



Министерство науки и высшего образования Российской Федерации
Федеральное государственное
бюджетное образовательное учреждение высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ

«Фундаментальные Науки»

КАФЕДРА

ФН-12 «Математическое моделирование»

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ НА ТЕМУ:

Обход графовых структур

Студент:

Подобедов В. В.

дата, подпись

Ф.И.О.

Преподаватель:

Волкова Л. Л.

дата, подпись

Ф.И.О.

Москва, 2023

Содержание

Введение	3
1 Аналитическая часть	4
2 Конструкторская часть	5
2.1 Обход бинарного дерева в ширину	5
2.2 Обход бинарного дерева в глубину	5
2.3 Написание бинарного дерева	5
3 Технологическая часть	7
4 Исследовательская часть	11
Заключение	13
Список используемых источников	14

Введение

Цель лабораторной работы: реализовать алгоритмы обхода графовых структур.

Для достижения поставленной цели требуется решить следующие **задачи**.

1. Описать граф и бинарное дерево.
2. Реализовать графовую структуру.
3. Реализовать возможность пользователю самостоятельно заполнять графовую структуру.
4. Реализовать обход графовой структуры в ширину и в глубину.
5. Выполнить тестирование реализации разработанного алгоритма.

Согласно варианту, требуется разработать бинарное дерево.

1 Аналитическая часть

Граф — это математическая абстракция, представляющая собой набор вершин и рёбер между этими вершинами.

Граф с петлями — это граф, в котором рёбра могут соединять вершины сами с собой, создавая так называемые "петли". Петля представляет собой ребро, начало и конец которого находятся в одной и той же вершине.

Граф без петель — это граф, в котором отсутствуют рёбра, соединяющие вершину с самой собой.

Бинарное дерево — это иерархическая структура данных в виде дерева, в которой каждый узел может иметь не более двух дочерних узлов: левый и правый. У дерева есть две главные характеристики:

1. **Корень** — верхний узел дерева, от которого начинаются все другие узлы. У бинарного дерева может быть только один корень.
2. **Листья** — узлы без дочерних узлов называются листьями. Листья находятся на самом нижнем уровне дерева.

2 Конструкторская часть

2.1 Обход бинарного дерева в ширину

Обход бинарного дерева в ширину происходит по уровням дерева: сначала посещаются все узлы на одном уровне, затем переходят к узлам следующего уровня. Для этого используется структура данных очередь (queue), которая помогает сохранять порядок обработки узлов.

Алгоритм обхода бинарного дерева в ширину:

1. Начать с корневого узла дерева и поместить его в очередь.
2. Пока очередь не пуста:
 - Извлечь узел из начала очереди.
 - Посетить этот узел и напечатать его значение.
 - Если у узла есть левый потомок, поместить его в конец очереди.
 - Если у узла есть правый потомок, поместить его в конец очереди.
3. Повторить шаг 2, пока очередь не опустеет.

2.2 Обход бинарного дерева в глубину

Обход бинарного дерева в глубину начинается с корневого узла и идет вглубь структуры дерева, прежде чем вернуться обрабатывать более поздние узлы. Для этого используется структура данных стек (stack), которая помогает сохранять порядок обработки узлов.

Алгоритм обхода бинарного дерева в ширину:

1. Начать с корневого узла дерева и поместить его в стек.
2. Пока стек не пуст:
 - Извлечь узел из начала стека.
 - Посетить этот узел и напечатать его значение.
 - Если у узла есть правый потомок, поместить его в конец стека.
 - Если у узла есть левый потомок, поместить его в конец стека.
3. Повторить шаг 2, пока стек не опустеет.

2.3 Написание бинарного дерева

С помощью шаблонных классов “BinaryTree”, который описывает саму структуру бинарного дерева, и “BinaryTreeNode”, который описывает узел бинарного дерева, реализовать бинарное дерево на ссылках. При этом нужно учесть, что левый или правый потомок могут отсутствовать. Требуется разработать следующие методы.

- Рекурсивный метод “addElement” — метод, который добавляет элемент в бинарное дерево.
- Рекурсивный метод “BreadthTree” — метод обхода бинарного дерева в ширину.
- Рекурсивный метод “DepthTree” — метод обхода бинарного дерева в глубину.
- Метод “addElementToTree” — метод, который обрабатывает введенный пользователем элемент и вызывает рекурсивный метод “addElement”.
- Метод “printBreadthTree” — метод, который выводит в консоль результат обхода дерева в ширину.
- Метод “printDepthTree” — метод, который выводит в консоль результат обхода дерева в глубину.

К программе предъявлены следующие требования. Пользователь самостоятельно вводит данные в бинарное дерево, после чего выводится результат обхода в ширину и глубину пользовательского дерева.

3 Технологическая часть

Лабораторная работа была реализована на языке программирования C++. Кроме стандартной библиотеки `<iostream>`, которая предоставляет возможность пользоваться средствами ввода-вывода для стандартной консоли, были подключены библиотеки `<queue>` и `<stack>`. Они представляют собой контейнеры, реализующие структуры данных “стек” и “очередь” соответственно, предоставляя удобные интерфейсы для хранения и управления данными в виде стека или очереди. Также была подключена библиотека `<string>` для пользования строками. Был также написан стек на ссылках.

Далее приведен листинг реализации стека (листинг 1).

Листинг 1 – Исходный код программы для реализации стека

```
#include <iostream>
#include <string>
#include <queue>
#include <stack>

using namespace std;

template <typename T>
class BinaryTreeNode {
public:
    T data;
    BinaryTreeNode<T>* left;
    BinaryTreeNode<T>* right;

    BinaryTreeNode(T value) : data(value), left(nullptr), right(
        nullptr) {}
};

template <typename T>
class BinaryTree {
private:
    BinaryTreeNode<T>* root;

    void addElement(BinaryTreeNode<T>* node) {
        string choice;

        cout << "Добавить значение в левый узел для узла " << node->
            data << "? (yes/no): ";
        cin >> choice;
```

```

        if (choice == "yes") {
            T value;
            cout << "Введите значение для левого узла : ";
            cin >> value;
            node->left = new BinaryTreeNode<T>(value);
            addElement(node->left);
        }
        else if (choice == "no") {
            node->left = nullptr;
        }

        cout << "Добавить значение в правый узел для узла " << node->
            data << "? (yes/no): ";
        cin >> choice;

        if (choice == "yes") {
            T value;
            cout << "Введите значение для правого узла : ";
            cin >> value;
            node->right = new BinaryTreeNode<T>(value);
            addElement(node->right);
        }
        else if (choice == "no") {
            node->right = nullptr;
        }
    }

    void BreadthTree(BinaryTreeNode<T>* node) {
        if (node == nullptr) {
            return;
        }

        queue<BinaryTreeNode<T>*> nodesQueue;
        nodesQueue.push(node);

        while (!nodesQueue.empty()) {
            BinaryTreeNode<T>* current = nodesQueue.front();
            nodesQueue.pop();
            cout << current->data << " ";

            if (current->left != nullptr) {
                nodesQueue.push(current->left);
            }
        }
    }

```



```

        }
        if (current->right != nullptr) {
            nodesQueue.push(current->right);
        }
    }
}

void DepthTree(BinaryTreeNode<T>* node) {
    if (node == nullptr) {
        return;
    }

    stack<BinaryTreeNode<T>*> nodesStack;
    nodesStack.push(node);

    while (!nodesStack.empty()) {
        BinaryTreeNode<T>* current = nodesStack.top();
        nodesStack.pop();
        std::cout << current->data << " ";

        if (current->right != nullptr) {
            nodesStack.push(current->right);
        }
        if (current->left != nullptr) {
            nodesStack.push(current->left);
        }
    }
}

```

public:

```

BinaryTree() : root(nullptr) {}

void addElementToTree() {
    T value;
    cout << "Введите значение для корня дерева : ";
    cin >> value;
    root = new BinaryTreeNode<T>(value);
    addElement(root);
}

void printBreadthTree() {
    cout << "Обход дерева в ширину : ";
}

```

```

        BreadthTree(root);
        cout << endl;
    }

    void printDepthFirstTree() {
        cout << "Обход дерева в глубину : ";
        DepthTree(root);
        cout << endl;
    }
};

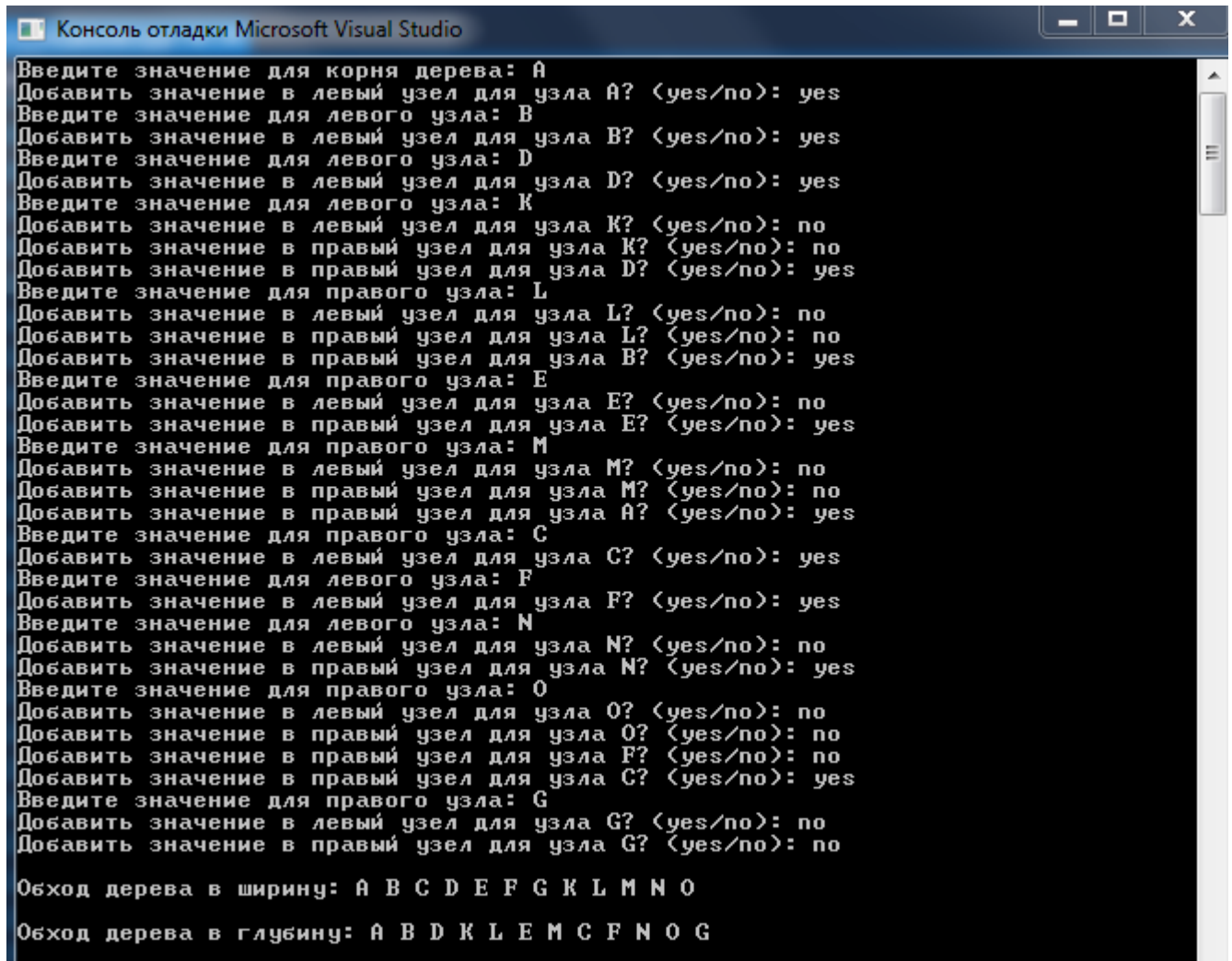
int main() {
    setlocale(0, "");
    BinaryTree<int> binaryTree;
    binaryTree.addElementToTree();
    cout << endl;
    binaryTree.printBreadthTree();
    cout << endl;
    binaryTree.printDepthFirstTree();
    return 0;
}

```

4 Исследовательская часть

Далее на рисунках представлен результат работы программы и входной файл с инфиксным выражением.

Результат работы программы, где пользователь вводит бинарное дерево, и программа выводит обход пользовательского дерева в ширину и глубину. Результат приведён на рис. 1.

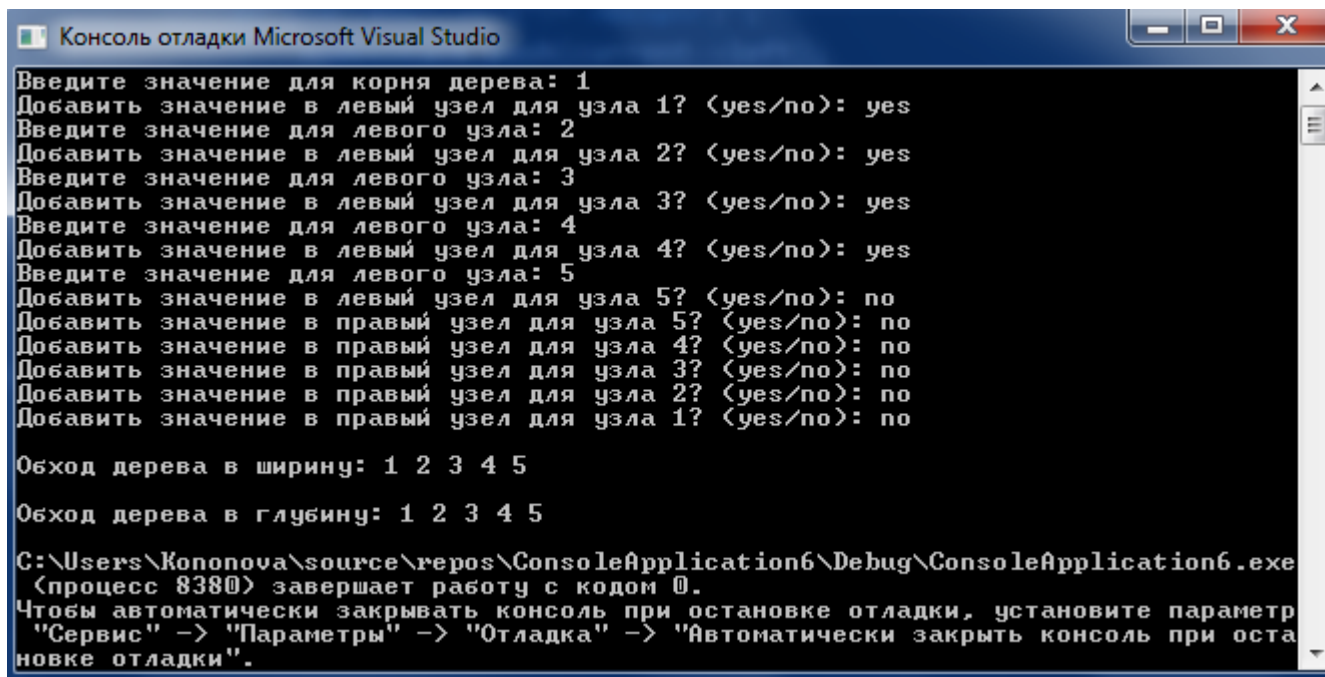


```
Консоль отладки Microsoft Visual Studio
Введите значение для корня дерева: A
Добавить значение в левый узел для узла A? (yes/no): yes
Введите значение для левого узла: B
Добавить значение в левый узел для узла B? (yes/no): yes
Введите значение для левого узла: D
Добавить значение в левый узел для узла D? (yes/no): yes
Введите значение для левого узла: K
Добавить значение в левый узел для узла K? (yes/no): no
Добавить значение в правый узел для узла K? (yes/no): no
Добавить значение в правый узел для узла D? (yes/no): yes
Введите значение для правого узла: L
Добавить значение в левый узел для узла L? (yes/no): no
Добавить значение в правый узел для узла L? (yes/no): no
Добавить значение в правый узел для узла B? (yes/no): yes
Введите значение для правого узла: E
Добавить значение в левый узел для узла E? (yes/no): no
Добавить значение в правый узел для узла E? (yes/no): yes
Введите значение для правого узла: M
Добавить значение в левый узел для узла M? (yes/no): no
Добавить значение в правый узел для узла M? (yes/no): no
Добавить значение в правый узел для узла A? (yes/no): yes
Введите значение для правого узла: C
Добавить значение в левый узел для узла C? (yes/no): yes
Введите значение для левого узла: F
Добавить значение в левый узел для узла F? (yes/no): yes
Введите значение для левого узла: N
Добавить значение в левый узел для узла N? (yes/no): no
Добавить значение в правый узел для узла N? (yes/no): yes
Введите значение для правого узла: O
Добавить значение в левый узел для узла O? (yes/no): no
Добавить значение в правый узел для узла O? (yes/no): no
Добавить значение в правый узел для узла F? (yes/no): no
Добавить значение в правый узел для узла C? (yes/no): yes
Введите значение для правого узла: G
Добавить значение в левый узел для узла G? (yes/no): no
Добавить значение в правый узел для узла G? (yes/no): no

Обход дерева в ширину: A B C D E F G K L M N O
Обход дерева в глубину: A B D K L E M C F N O G
```

Рис. 1 – Пример №1

Результат работы программы, где пользователь вводит бинарное дерево, состоящее только из левых потомков, и программа выводит обход пользовательского дерева в ширину и глубину. Результат приведён на рис. 2.



```
Консоль отладки Microsoft Visual Studio
Введите значение для корня дерева: 1
Добавить значение в левый узел для узла 1? <yes/no>: yes
Введите значение для левого узла: 2
Добавить значение в левый узел для узла 2? <yes/no>: yes
Введите значение для левого узла: 3
Добавить значение в левый узел для узла 3? <yes/no>: yes
Введите значение для левого узла: 4
Добавить значение в левый узел для узла 4? <yes/no>: yes
Введите значение для левого узла: 5
Добавить значение в левый узел для узла 5? <yes/no>: no
Добавить значение в правый узел для узла 5? <yes/no>: no
Добавить значение в правый узел для узла 4? <yes/no>: no
Добавить значение в правый узел для узла 3? <yes/no>: no
Добавить значение в правый узел для узла 2? <yes/no>: no
Добавить значение в правый узел для узла 1? <yes/no>: no

Обход дерева в ширину: 1 2 3 4 5
Обход дерева в глубину: 1 2 3 4 5

C:\Users\Kononova\source\repos\ConsoleApplication6\Debug\ConsoleApplication6.exe
(процесс 8380) завершает работу с кодом 0.
Чтобы автоматически закрывать консоль при остановке отладки, установите параметр
"Сервис" -> "Параметры" -> "Отладка" -> "Автоматически закрыть консоль при оста
новке отладки".
```

Рис. 2 – Пример №2

Заключение

Цель лабораторной работы достигнута: реализованы алгоритмы обхода бинарного дерева. В результате выполнения лабораторной работы были выполнены все задачи.

1. Описано бинарное дерево.
2. Программно реализовано бинарное дерево.
3. Реализована возможность пользователю самостоятельно заполнять бинарное дерево.
4. Реализован обход бинарного дерева в ширину и в глубину.
5. Выполнено тестирование реализации разработанного алгоритма.

Список литературы

1. Рафгарден Тим. Совершенный алгоритм. Графовые алгоритмы и структуры данных. — СПб.: Питер, 2019. — 256 с.: ил. — (Серия “Библиотека программиста”).